

Comparing DQN and Double DQN for Spacecraft Landing Control in LunarLander-v3

Jess Barrett (EID: 854433853)
CIS 530 Spring 2026
Kansas State University

Problem & Motivation

The Problem

- Land a spacecraft safely in a continuous, 8D state space
- 4 discrete thruster actions, delayed feedback
- Hand-coded controllers are weak, no labeled dataset for supervised learning

Why RL?

- Agent learns by trial-and-error in a simulator
- Standard benchmark for value-based deep RL
- "Solved" = average reward 200+ over 100 episodes

Track: Search, Games, & Constraint Satisfaction

Pillar: Reinforcement Learning

Research Question

Does Double DQN beat vanilla DQN on LunarLander-v3?

- DDQN is supposed to fix DQN's overestimation bias (van Hasselt, AAAI/2016)
- In theory: DDQN should produce more stable, higher-quality policies
- In practice: does the theoretical advantage actually translate to better empirical performance at this scale?

Plus a sanity check: do either of these methods beat a random baseline?

Background: DQN

Deep Q-Network (Mnih et al., *Nature* 2015)

- Neural net approximates $Q^*(s, a)$ for continuous state spaces
- Two stability tricks:
 - **Experience replay**: store transitions, train on random batches
 - **Target network**: separate frozen copy gives stable training targets
- Update rule: $y = r + \gamma \cdot \max_{a'} Q_{\text{target}}(s', a')$

Background: Double DQN

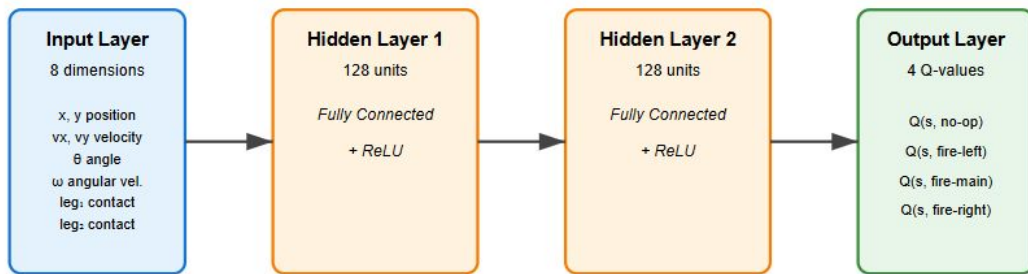
The fix DDQN introduces (van Hasselt et al., AAAI 2016)

- DQN's max operator overestimates Q-values
- DDQN decouples action **selection** from action **evaluation**:
 - Online net picks the action
 - Target net evaluates its Q-value
- $y = r + \gamma \cdot Q_{\text{target}}(s', \operatorname{argmax}_{\{a\}} Q_{\text{online}}(s', a'))$
- One-line change. Theoretically reduces overestimation bias.

Architecture & Hyperparameters

Q-Network (same for both methods)

- Input: 8D state $\rightarrow$ FC(128) + ReLU $\rightarrow$ FC(128) + ReLU $\rightarrow$ 4 Q-values



Identical training setup:

- Replay buffer: 100k | Batch: 64 | γ : 0.99
- Adam (lr=1e-3) | MSE loss | Target sync every 10 eps
- ϵ -greedy: 1.0 $\rightarrow$ 0.01 (decay 0.995/ep)
- 600 episodes per trial

Experimental Protocol

The setup that allows for fair comparison:

- 5 independent trials of each method (different random seeds)
- After training: evaluate **greedily** ($\epsilon = 0$) on **100 fixed evaluation episodes**
- Same evaluation seeds across all methods $\rightarrow$ comparisons are **paired**
- Random baseline evaluated on the same seeds

Why this matters: RL is famously seed-sensitive. Single-run comparisons can lie.
(*Henderson et al., AAAI 2018*)

Hypotheses & Tests

Three null hypotheses ($\alpha = 0.05$):

$H_0^1: \mu(\text{DQN}) = \mu(\text{Random})$ $H_0^2: \mu(\text{DDQN}) = \mu(\text{Random})$ $H_0^3: \mu(\text{DQN}) = \mu(\text{DDQN})$

Two tests per comparison:

- Paired t-test (parametric, primary)
- Wilcoxon signed-rank test (non-parametric, sanity check)

Plus Cohen's d — because effect size > significance alone in deep RL

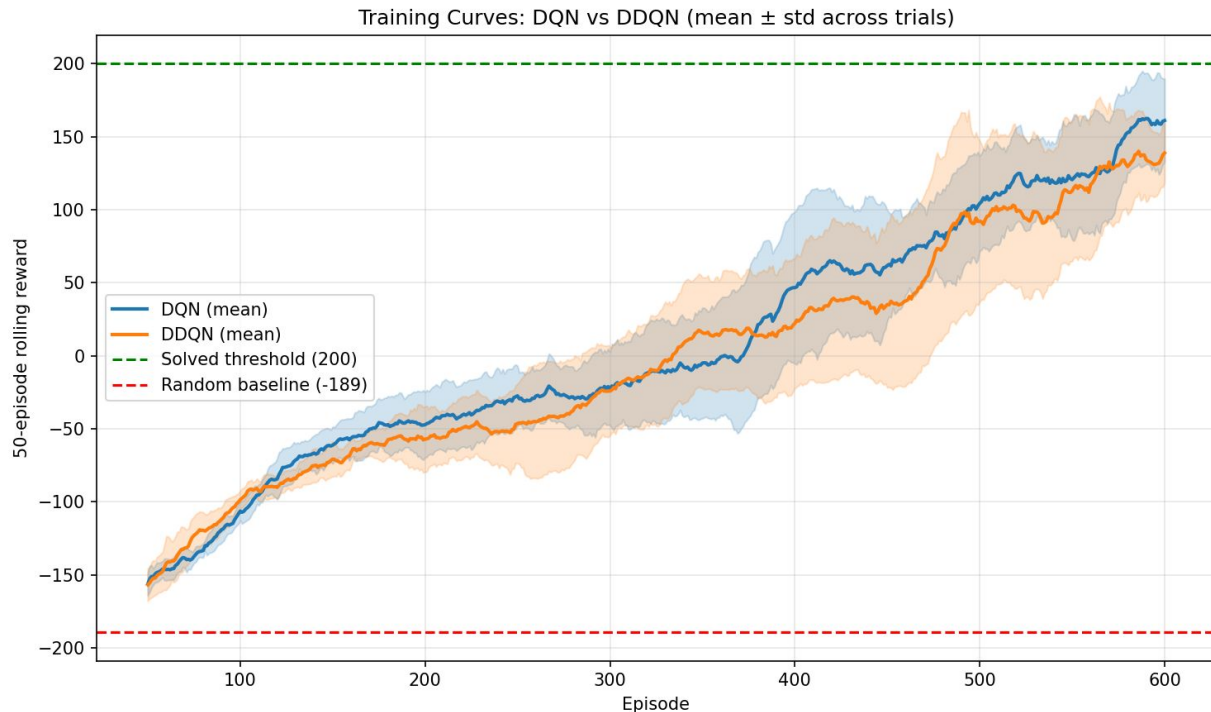
Results: Training Curves

Both methods start near random baseline (≈ -190)

DQN's mean climbs faster and ends higher

DDQN trails but is still trending upward at episode 600

Wide bands = high inter-trial variance



Results: Statistical Tests

Summary

Method	Mean	Std
Random	-189.4	7.9
DQN	151.5	37.9
DDQN	117.6	71.2

Hypothesis tests (paired t-test):

			Comparison	p	Cohen's d	Decision
Random	-189.4	7.9	DQN vs Random	<0.001	11.13	Reject H_0
DQN	151.5	37.9	DDQN vs Random	0.0012	5.42	Reject H_0
DDQN	117.6	71.2	DQN vs DDQN	0.4551	-0.53	Fail to reject

The Surprising Finding

DDQN did *not* beat DQN.

- Point estimate slightly favored vanilla DQN (151.5 vs 117.6)
- $p = 0.46 \rightarrow$ no statistically significant difference
- Effect size $d = -0.53$ (small, in DQN's favor)

Why?

- LunarLander only has **4 actions** $\rightarrow$ less max-bias for DDQN to fix
- 5 trials is **underpowered** — single bad seed (DDQN trial 4 = 5.4) shifts the mean
- Some DDQN trials still climbing at episode 600 — may not have fully converged

Failure Modes

Three things that went wrong worth talking about:

1. **Seed sensitivity:** DQN trial 1 = 94.3, trial 3 = 204.0 (same algorithm, different seed)
2. **Incomplete convergence:** DDQN curves still climbing at 600 episodes
3. **Train/eval mismatch:** agents that look great with ϵ -greedy can fail under greedy evaluation
 - DQN trial 4: training avg 179.1 $\rightarrow$ eval 125.7

These failure modes echo Henderson et al.'s reproducibility concerns (AAA/2018).

Reflection Q1

What was most technically challenging?

Understanding why DDQN didn't beat DQN despite the theory.

- The literature said DDQN should win — and it didn't
- Had to dig into *why*: small action space, low trial count, incomplete convergence, seed variance
- Forced me to think about the gap between **what an algorithm should do in theory** vs **what it does in this specific setup**
- The honest answer turned out to be more interesting than the expected one

Reflection Q2

What did I learn the most about?

Reinforcement learning from the ground up, what DQN actually is and how it works.

- Came in knowing basically nothing about RL — I'd never trained an agent before
- Had to learn the whole framework: states, actions, rewards, Q-values, the Bellman equation
- Then implementing DQN myself meant figuring out *why* each piece exists. Replay buffer, target network, epsilon-greedy
- Now I actually get how an agent learns from experience

Reflection Q3 + Future Work

Top priority if I continued:

Run more trials ($N \geq 30$) for tight confidence intervals on DQN vs DDQN.

- 5 trials wasn't enough to settle the comparison
- One bad seed dominated DDQN's average
- With 30+ trials, the underlying difference (if any) would be clear

Other directions:

- Prioritized experience replay
- Dueling architectures
- Longer training budgets
- Generalization to perturbed environments

Summary

Findings

- Both DQN and DDQN beat random baseline ($p < 0.005$, $d > 5$)
- DDQN did **not** significantly beat DQN ($p = 0.46$)
- RL results are highly seed-sensitive at small N

Honest conclusion: the theoretical advantage didn't show up empirically *in this setup, at this scale*.

Repository: github.com/jess-barrett/LunarLander-DQN